

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical and Computer Engineering



AI & Machine Learning Lab

Experiment No. 14: Implementing Convolutional Neural Network (CNN) for Edge-Based Mask Detection and Face Recognition

Name of Student: ...[Bushra Nazir](#).....

Registration No.: ...[1071-F22](#).....

Date of Experiment: ...[1-6-2026](#).....

Submitted To: ...[Engr.Asad](#).....

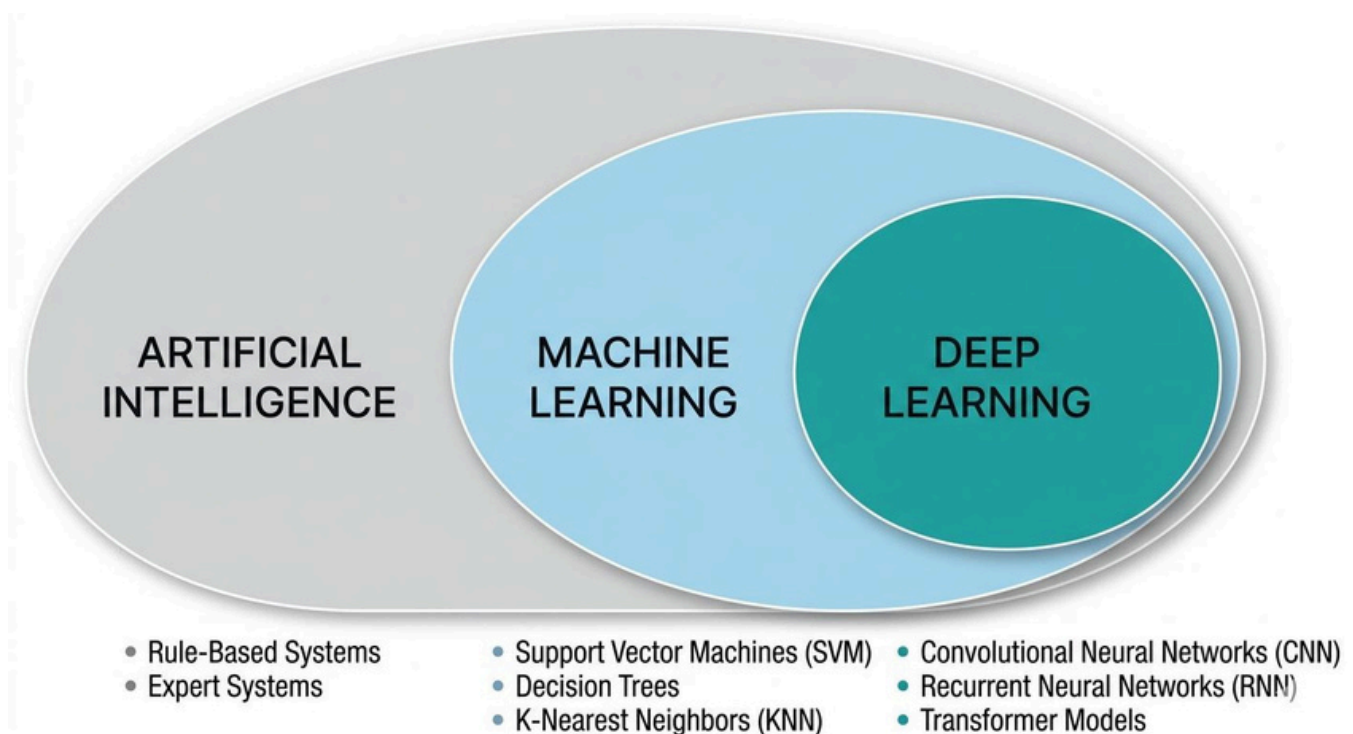
Objectives

- To understand the working principles of Convolutional Neural Networks (CNNs) and their role in image classification and feature extraction.
- To implement a CNN model for mask detection system.
- To Implement a CNN model for Attendance system

Theoretical Background

What is Deep Learning?

Deep Learning is a subfield of Machine Learning that uses artificial neural networks with multiple hidden layers to learn hierarchical representations of data. Instead of manual engineering features, deep learning models automatically discover the patterns needed for classification or prediction by training on large datasets.

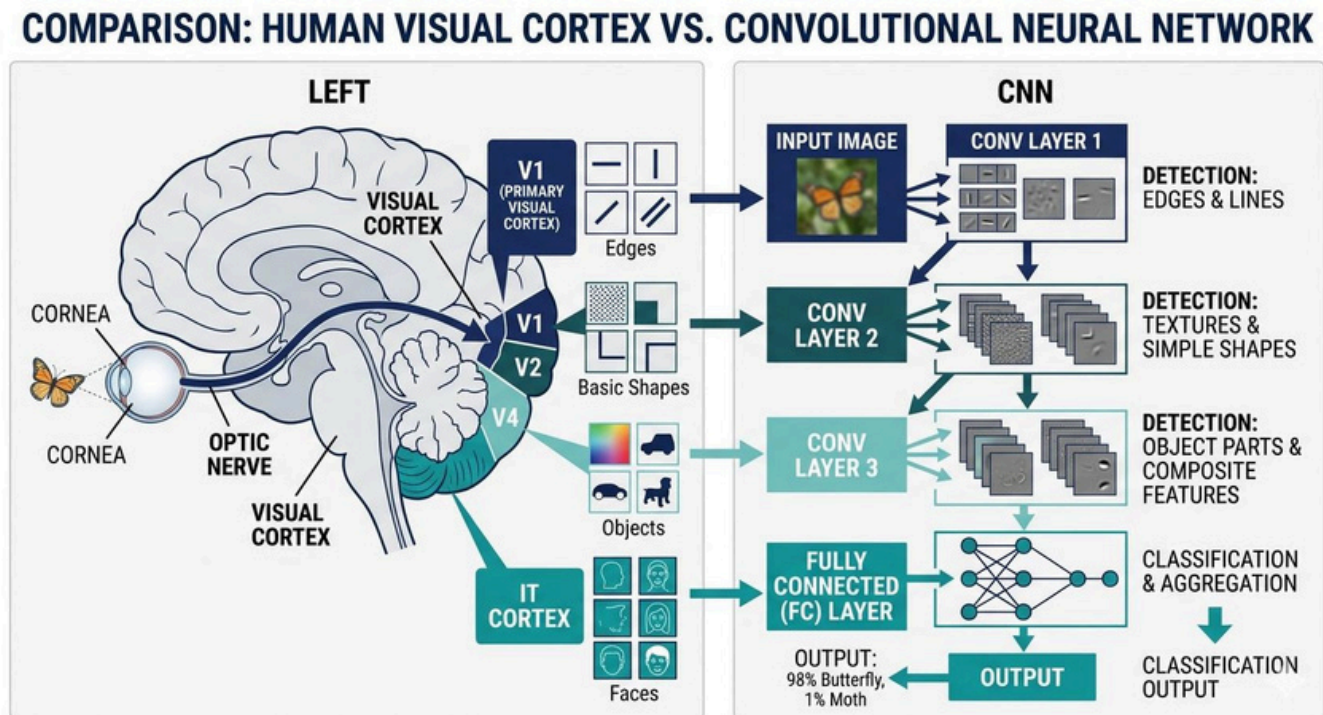


The Human Visual Cortex Inspiration for CNN

The brain's visual processing begins in the primary visual cortex (V1), which detects simple patterns such as edges and orientations. Subsequent layers (V2, V4, IT cortex) progressively detect more complex

shapes corners, objects, and finally full scenes. Each neuron responds to a limited region of the visual field called its receptive field.

CNNs mirror this architecture: early convolutional layers detect edges and textures; deeper layers combine these into increasingly complex semantic features. This biological inspiration is why CNNs excel at visual tasks.



Why Not Regular ANN for Images?

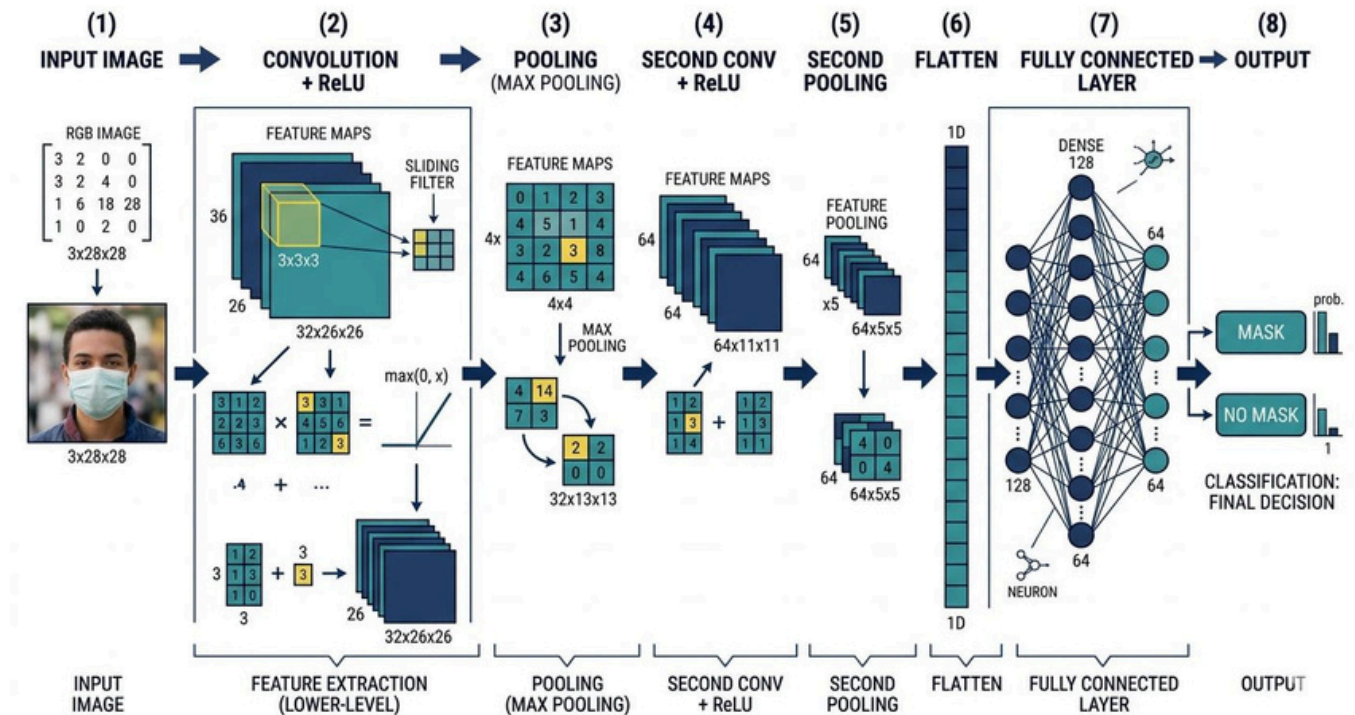
Consider a modest 224×224 RGB image it contains $224 \times 224 \times 3 = 150,528$ pixel values. Connecting every pixel to every neuron in the first hidden layer (say, 1,000 neurons) requires 150 million parameters in just one layer. This creates three fundamental problems:

Problem	Explanation
Computational Cost	Hundreds of millions of weights → slow training, huge memory requirements.
Spatial Blindness	ANN treats each pixel independently it ignores the 2D spatial structure and relationships between neighbouring pixels.
Location Sensitivity	If the same object (e.g., a cat) appears in a different part of the image, ANN may fail to recognise it because it learns position-specific patterns.

Overfitting Risk	Too many parameters with limited data causes the model to memorise rather than generalise.
-------------------------	--

Components of a CNN

A CNN is a pipeline of specialized layers, each performing a distinct operation on the data. The standard architecture is:

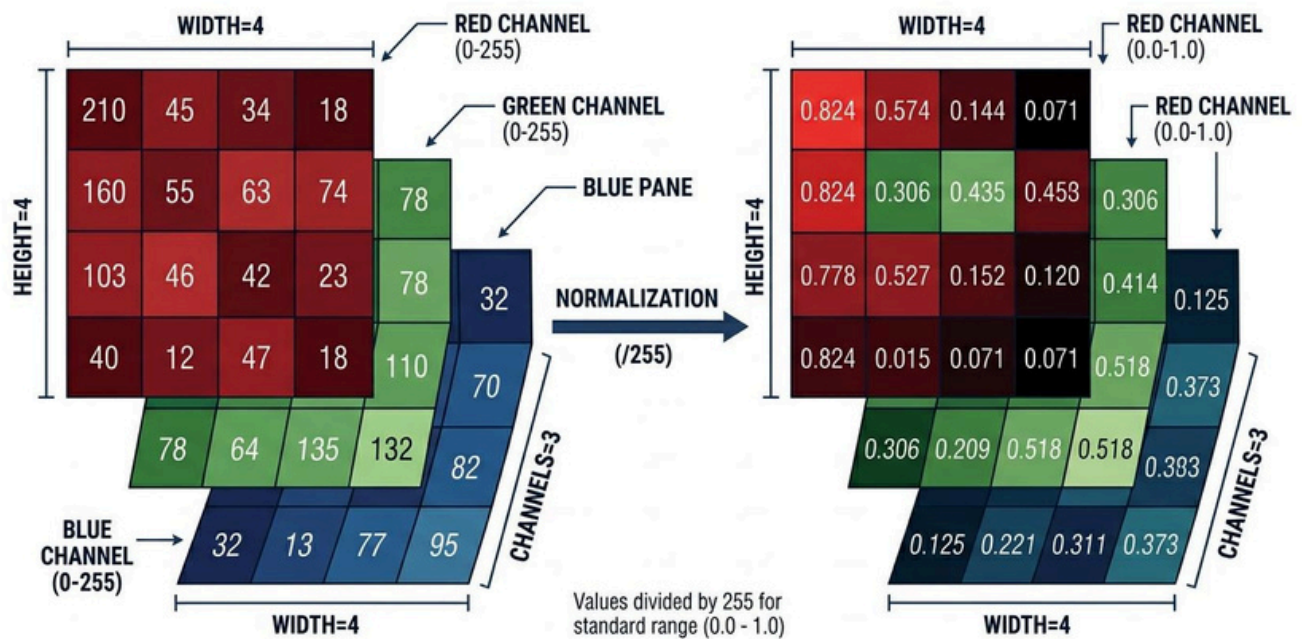


Input Layer

The input to a CNN is a 3-dimensional tensor of shape [Height × Width × Channels]. For color images, Channels = 3 (Red, Green, Blue). For grayscale, Channels = 1.

Normalization: Pixel values range from 0 to 255. Before feeding into the network, we normalize them to [0, 1] by dividing by 255.0. This prevents large gradients and speeds up training.

Property	Grayscale	RGB (Colour)
Channels	1	3
Example shape	(64, 64, 1)	(64, 64, 3)
Total pixel values	4,096	12,288
Typical use	Document scan, X-ray	Photos, videos



Convolution Layer

This is the core building block of a CNN. A small matrix called a **filter (or kernel)** slides across the input image. At each position, it performs an element-wise multiplication between the filter and the image patch it covers, then sums all the results to produce a single output value. This produces a **Feature Map** (also called an activation map).

How Convolution Works Step by Step

Let us work through a concrete numerical example.

Input image (6×6): (simplified, single channel)

```

1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1
1 0 1 0 1 0
0 1 0 1 0 1

```

Filter / Kernel (3×3) Vertical Edge Detector:

```

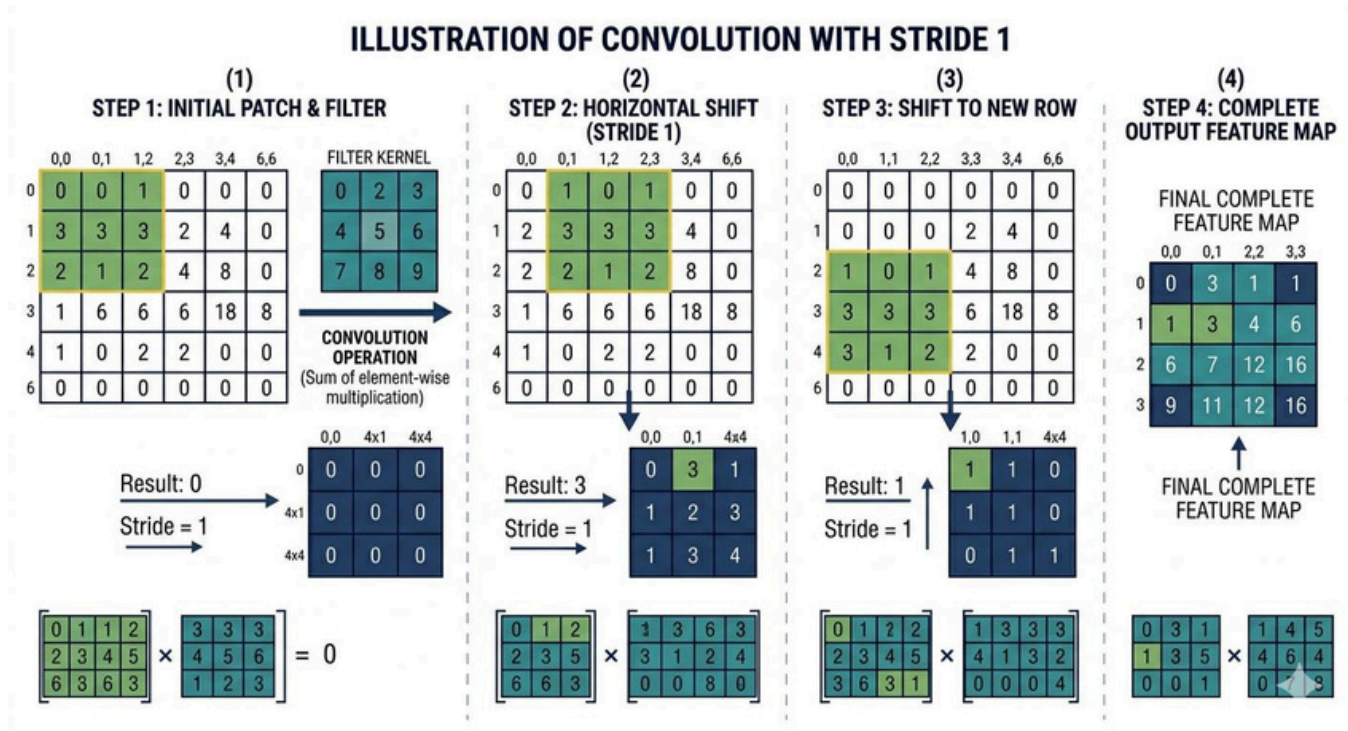
1 0 -1
1 0 -1

```

1 0 -1

The filter slides across the image with a stride of 1. At the first position (top-left):

Patch: Filter: Element-wise $\times$ then sum:
 1 0 1 0 10-1 $(1 \times 1) + (0 \times 0) + (1 \times -1)$
 1 0 1 0 10-1 $= + (0 \times 1) + (1 \times 0) + (0 \times -1)$
 1 10-1 $(1 \times 1) + (0 \times 0) + (1 \times -1)$
 $= 1 + 0 - 1 + 0 + 0 + 0 + 1 + 0 - 1 = 0$



2Output Size Formula

Given an input of size $W \times W$ and a filter of size $F \times F$ with padding P and stride S :

Output Dimension Formula

Output Size = $\lfloor (W - F + 2P) / S \rfloor + 1$

Example: $W=6, F=3, P=0, S=1 \rightarrow \lfloor (6-3+0)/1 \rfloor + 1 = 4$

Result: 4x4 Feature Map ✓

Key Concepts

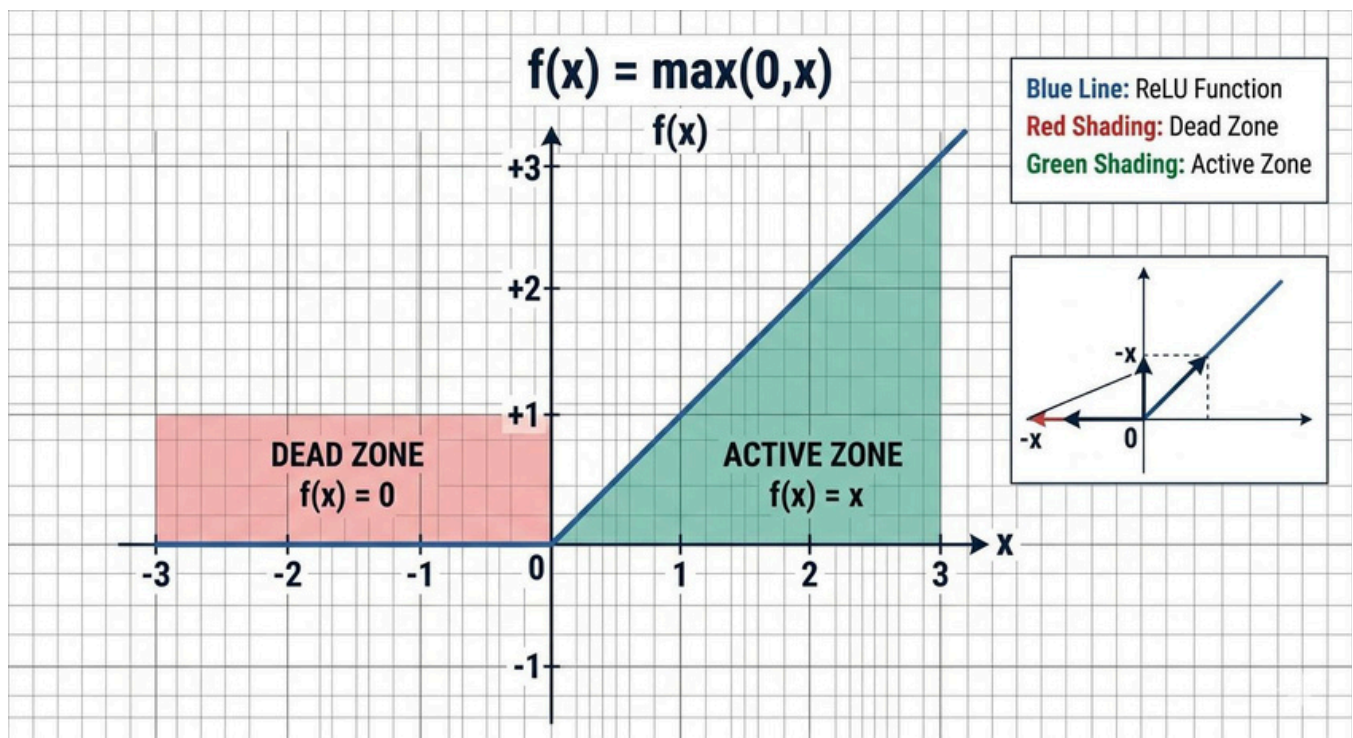
Term	Explanation
Filter / Kernel	Small weight matrix (e.g., 3×3 or 5×5) that learns to detect specific features. A CNN learns hundreds of filters automatically through backpropagation.
Stride	How many pixels the filter moves at each step. Stride=1 → dense, overlapping sampling. Stride=2 → downsampled output, fewer computations.
Padding	Adding zeros around the image border. 'Valid' padding (P=0) reduces output size. 'Same' padding keeps output the same size as input. Helps preserve edge information.
Feature Map	The output of applying one filter to the entire input. Multiple filters → multiple feature maps (one per filter), forming the depth of the next layer.
Parameter Sharing	The same filter weights are used at every spatial position. This dramatically reduces parameters vs ANN.

Activation Function ReLU

After the convolution, an activation function is applied elementwise to the feature map to introduce non-linearity. Without this, the entire network would simply be a linear transformation, no matter how many layers it had.

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$

Negative values become 0; positive values pass through unchanged. ReLU is preferred because it is computationally cheap, does not suffer from the vanishing gradient problem (unlike sigmoid/tanh), and empirically works very well for deep networks.

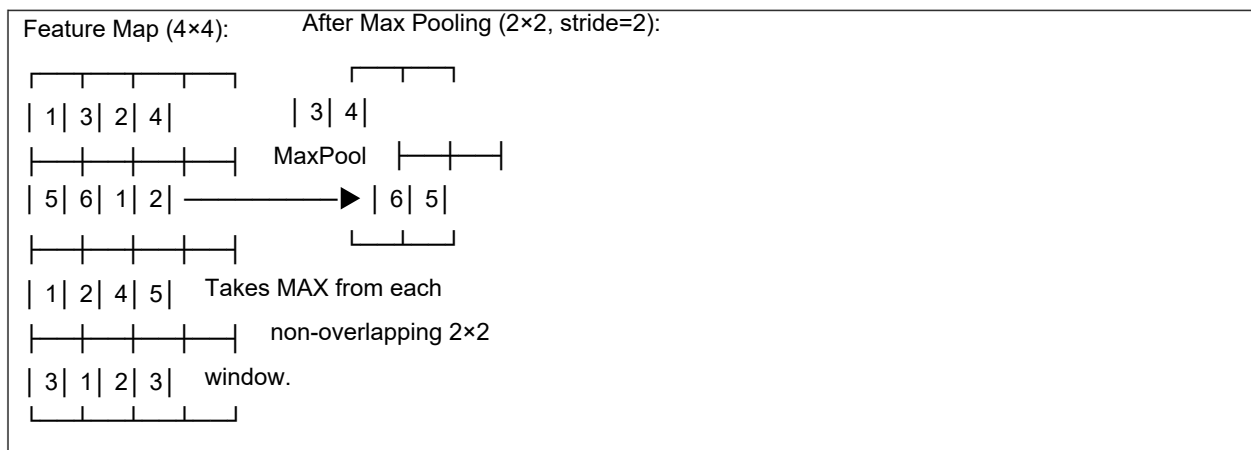


Pooling Layer

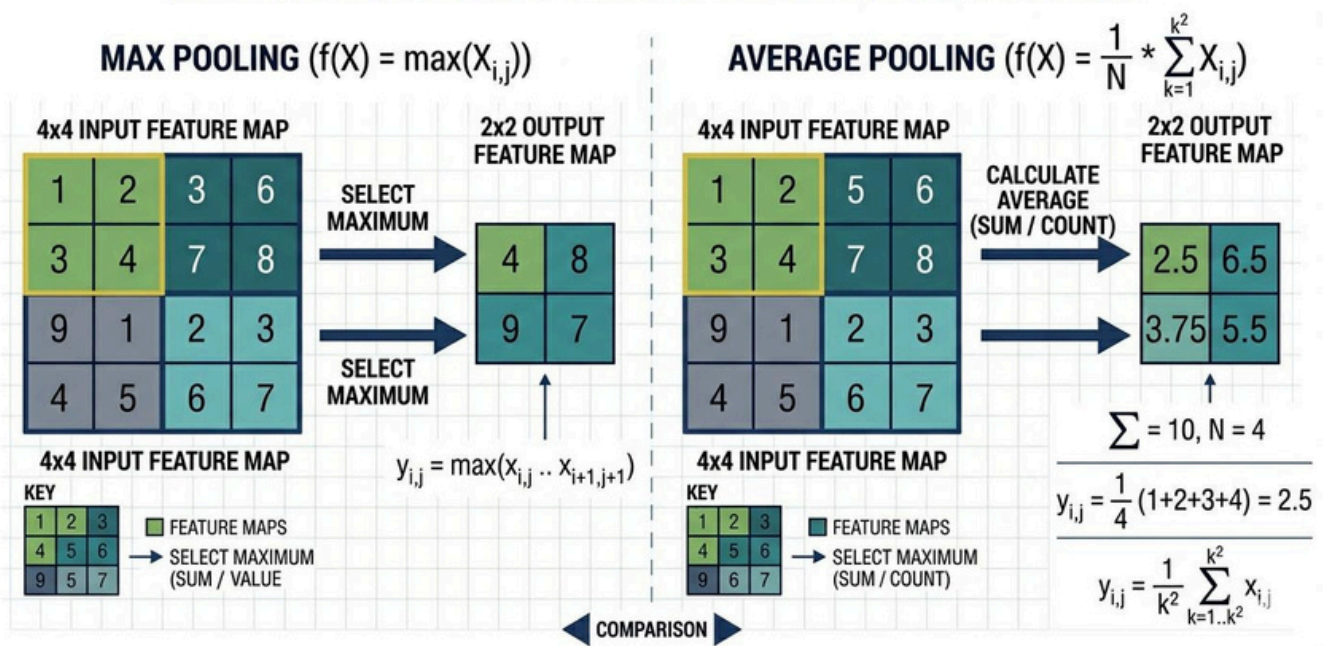
Pooling reduces the spatial dimensions (width and height) of the feature map while retaining the most important information. This achieves two things: (1) reduces computation in subsequent layers, and (2) provides translation invariance making the network less sensitive to small shifts in the object's position.

Max Pooling Worked Example

Using a 2×2 window with stride 2 on a 4×4 feature map:



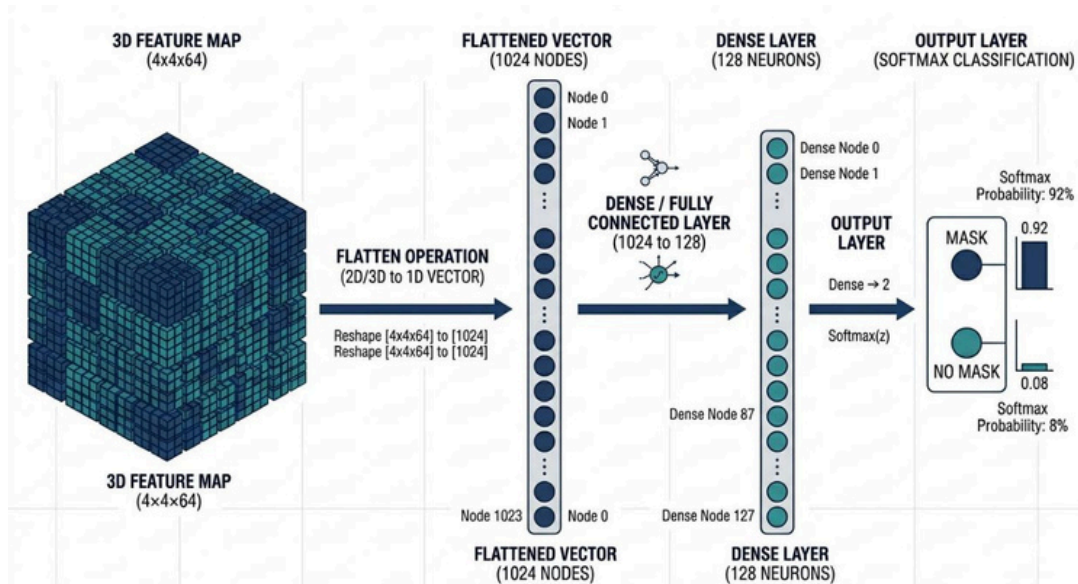
COMPARISON: MAX POOLING VS. AVERAGE POOLING



Fully Connected (Dense) Layer

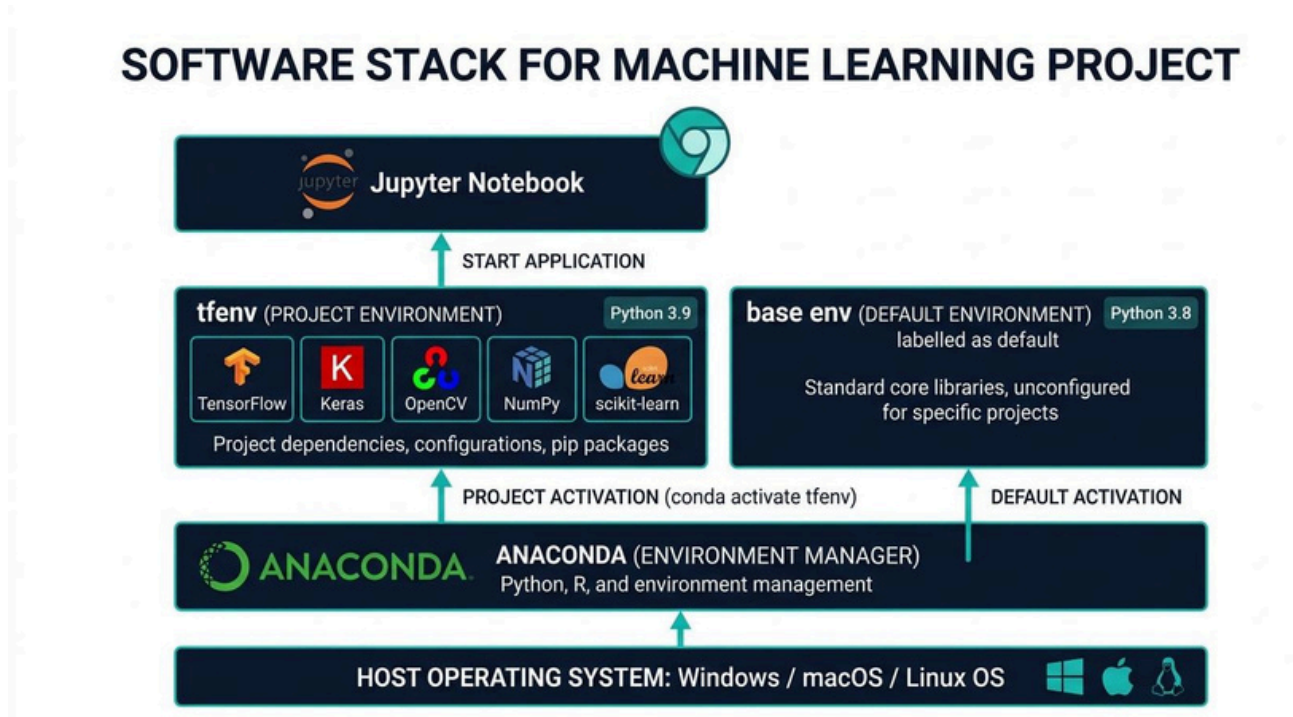
After the series of Conv+Pool layers, the 3D feature maps are flattened into a 1D vector and passed to one or more fully connected (Dense) layers. These are identical to standard ANN layers every neuron connects to every neuron in the next layer.

The FC layers act as the classifier, combining all the extracted spatial features to make the final prediction. The last layer uses a Softmax activation for multi-class classification, producing a probability distribution over the output classes.



Platform & Environment Setup

We will use the Anaconda ecosystem to create an isolated Python environment with GPU-ready TensorFlow, OpenCV, and Jupyter Notebook. This approach avoids library conflicts and mirrors professional ML development workflows.



3.1 Environment Setup Step by Step

Step 1 Install Anaconda

Download and install Anaconda from <https://www.anaconda.com/download>. Choose the version for your OS (Windows/macOS/Linux). During installation on Windows, check 'Add Anaconda to PATH' or use Anaconda Prompt for all commands.

Step 2 Create the TensorFlow Environment

Open Anaconda Prompt (Windows) or Terminal (macOS/Linux) and run:

```
# Create a new isolated environment named 'tfenv' with Python 3.9
conda create -n tfenv python=3.9
```

```
# Activate the environment (must do this before installing packages)
conda activate tfenv
```

Step 3 Install Required Libraries
With the tfenv environment active, install all dependencies:

```
# Core deep learning framework
pip install tensorflow

# Computer vision + machine learning utilities
pip install opencv-python scikit-learn

# Jupyter Notebook for interactive development
pip install notebook

# Launch Jupyter Notebook
jupyter notebook
```

Library Reference

Library	Role	Key Functions Used
TensorFlow / Keras	Deep Learning framework	Sequential, Conv2D, MaxPooling2D, Dense, Dropout
OpenCV (cv2)	Image & video processing	VideoCapture, imread, resize, cvtColor, rectangle
NumPy	Numerical arrays	array(), reshape(), argmax()
scikit-learn	Dataset utilities	train_test_split
os	File system access	listdir(), makedirs(), path.join()
csv / datetime	Attendance logging	csv.writer(), datetime.now()

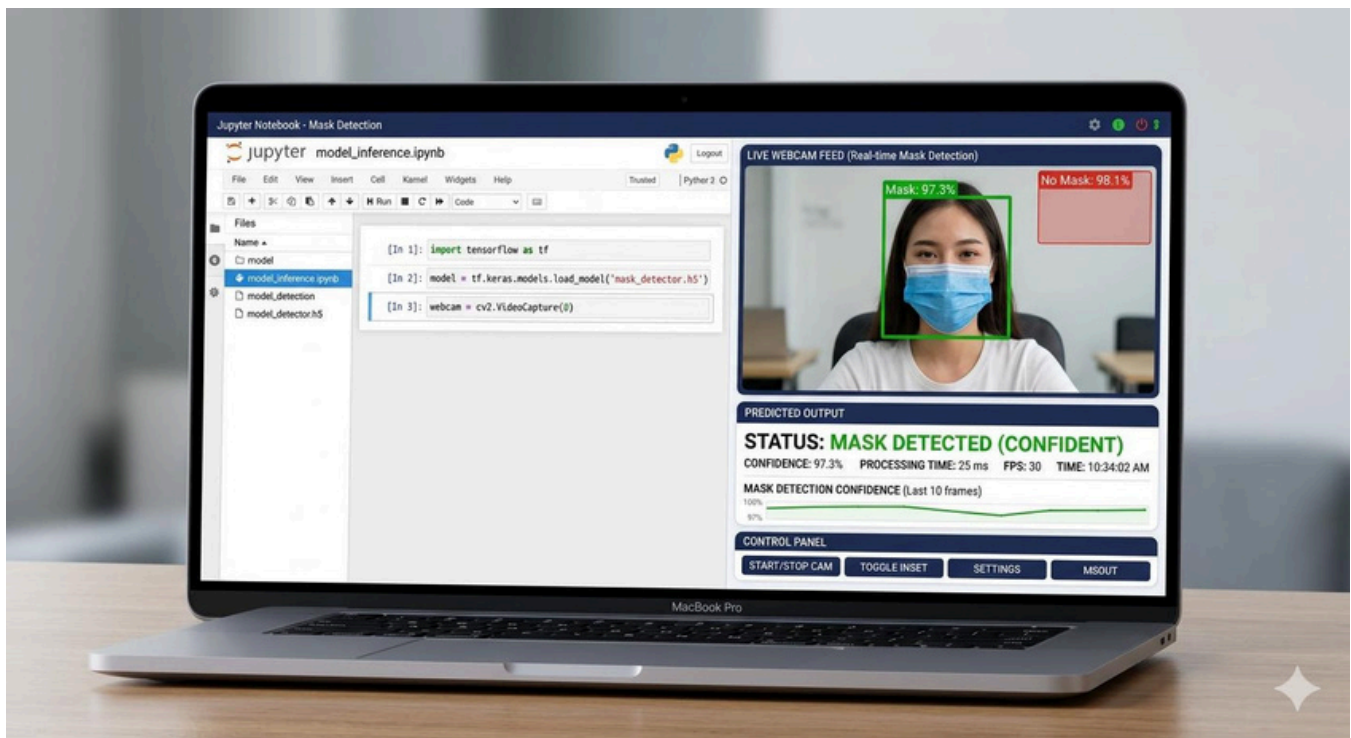
LAB TASKS

Task 1 Real-Time Mask Detection Using CNN

Task Objective

Train a CNN model to classify webcam frames as 'Mask' or 'No Mask' in real time. This simulates a practical computer vision application used in health safety monitoring systems.

Learning Outcomes: Build a CNN from scratch · Collect your own dataset · Train, save, and deploy a model · Apply real-time inference with OpenCV face detection.



Project Folder Structure

```
mask_detection/
├── dataset/
│   ├── with_mask/      ← 50 selfie images with mask
│   └── without_mask/   ← 50 selfie images without mask
├── mask_detector.keras ← Saved trained model (created after training)
├── haarcascade_frontalface_default.xml ← Download from OpenCV GitHub
├── 01_capture_dataset.ipynb
└── 02_train_model.ipynb
```

Where to get the Haar Cascade file

Download haarcascade_frontalface_default.xml from:

<https://github.com/opencv/opencv/tree/master/data/haarcascades>

Place it in the same folder as your detection notebook.

Step 1 Dataset Collection

You will collect your own dataset using the webcam. This teaches you the full ML pipeline: data collection → preprocessing → training → deployment.

```
import cv2
import os

# — Create dataset folder structure —————
classes = ["with_mask", "without_mask"]
for cls in classes:
    os.makedirs(f"dataset/{cls}", exist_ok=True)

# — Choose which class to capture —————
# Change this variable, then run the cell again for the second class
class_name = "with_mask" # Change to "without_mask" for second run

# — Start webcam —————
cap = cv2.VideoCapture(0) # 0 = default webcam
count = 0

print(f"Capturing '{class_name}' images.")
print("Press 's' to save a frame, 'q' to quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        print("Error: Cannot read from webcam.")
        break

    # Display live preview with frame counter
    cv2.putText(frame, f"{class_name} | Saved: {count}/50",
                (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, (0,255,0), 2)
```

```

cv2.imshow("Capture - Press 's' to Save, 'q' to Quit", frame)

key = cv2.waitKey(1)
if key == ord('s'):
    path = f"dataset/{class_name}/{class_name}_{count}.jpg"
    cv2.imwrite(path, frame)
    print(f" Saved: {path}")
    count += 1
elif key == ord('q') or count >= 50:
    break

cap.release()
cv2.destroyAllWindows()
print(f"Done! Collected {count} images for '{class_name}'")

```

Dataset Tips

- Vary your pose, angle, and lighting for both classes diversity improves generalisation.
- Ensure the mask covers your nose and mouth fully in the 'with_mask' shots.
- Aim for 50 images per class minimum. More data = better accuracy.
- Run the cell twice: once with class_name='with_mask', once with 'without_mask'.

Step 2 Data Preprocessing & Model Training

Now load, resize, normalise the images, split into train/test sets, build the CNN, and train it.

```

import os, cv2, numpy as np
from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

# — Load & preprocess dataset —————
IMG_SIZE = 64 # Resize all images to 64x64 pixels
CATEGORIES = ["with_mask", "without_mask"]

data, labels = [], []

```

```

for label, category in enumerate(CATEGORIES):
    folder_path = os.path.join("dataset", category)
    for filename in os.listdir(folder_path):
        img_path = os.path.join(folder_path, filename)
        img = cv2.imread(img_path)
        if img is not None:
            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            data.append(img)
            labels.append(label)

# Normalise pixel values from [0,255] → [0.0, 1.0]
X = np.array(data) / 255.0
y = to_categorical(labels) # One-hot encode: 0→[1,0], 1→[0,1]

#—Train/ Testsplit(80%train,20%test)———
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=labels)

print(f"Training samples : {len(X_train)}")
print(f"Testing samples : {len(X_test)}")

#—BuildCNNArchitecture———
model = Sequential([
    # Block 1: Detect simple features (edges, colours)
    Conv2D(32, (3, 3), activation='relu',
        input_shape=(IMG_SIZE, IMG_SIZE, 3)),
    MaxPooling2D(2, 2),

    # Block 2: Detect complex features (shapes, textures)
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D(2, 2),

    # Classifier head
    Flatten(),
    Dense(128, activation='relu'),
    Dropout(0.5), # Randomly zero 50% of neurons → prevent overfitting
    Dense(2, activation='softmax') # 2 classes: [with_mask, without_mask]
])

model.summary() # Print architecture overview

#—Compile & Train ———
model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

```

```

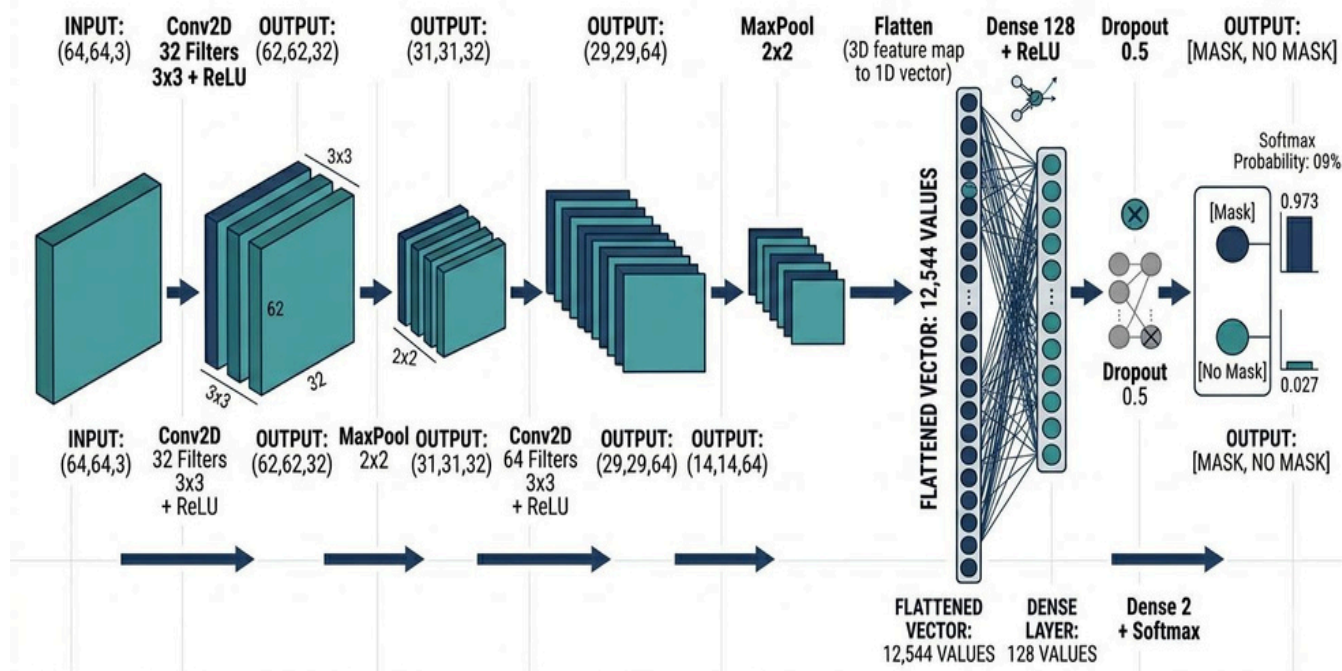
#EarlyStoppingstops    trainingif val_loss stops improving (saves time)
early_stop= EarlyStopping(monitor='val_loss', patience=3,
                           restore_best_weights=True)

history = model.fit(
    X_train, y_train,
    epochs=20,
    batch_size=16,
    validation_data=(X_test,y_test),
    callbacks=[early_stop]
)

#—Savethe    trained model—————
model.save("mask_detector.keras")
print("Model savedas  'mask_detector.keras'")

```

COMPLETE CNN ARCHITECTURE FOR FACE MASK DETECTION



Understanding Architecture Choices

Choice	Why?
Input 64×64	Balances detail vs. speed. Large enough to see facial features; small enough for fast training on CPU.
32 filters in Conv1	Learns 32 different low-level features (edges, colours) standard starting point.
64 filters in Conv2	Doubles capacity to learn more complex patterns in the deeper layer.
Dropout(0.5)	With only 100 images, overfitting is a real risk. Dropout randomly disables neurons during training, forcing the model to be robust.
Adam optimiser	Adaptive learning rate generally works well out-of-the-box without manual tuning.
Softmax output	Ensures the two output values sum to 1.0, interpretable as probabilities.

Step 3 Real-Time Mask Detection

The Haar Cascade classifier from OpenCV detects faces in each frame. Each detected face region is cropped, resized to 64×64, normalized, and fed to our trained CNN. **Note:** A bug exists in the original code the indentation of lines after the for loop is incorrect. The corrected version is below.

```
import cv2, numpy as np
from tensorflow.keras.models import load_model

# — Load model & setup —————
model = load_model("mask_detector.keras")
labels = ["Mask", "No Mask"]
IMG_SIZE = 64

# Load Haar Cascade for face detection
face_cascade = cv2.CascadeClassifier(
    cv2.data.harcascades + "haarcascade_frontalface_default.xml"
)
# Alternative: use local file
# face_cascade = cv2.CascadeClassifier("haarcascade_frontalface_default.xml")
```

```

if face_cascade.empty():
    raise IOError("Haar cascade file not found. Check the path.")

# — Start webcam —————
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise IOError("Cannot access webcam. Check connection.")

print("Real-time Mask Detection started. Press 'q' to quit.")

while True:
    ret, frame = cap.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    faces = face_cascade.detectMultiScale(
        gray, scaleFactor=1.1, minNeighbors=5, minSize=(60, 60))

    for (x, y, w, h) in faces:          # ← FIX: all code below is INSIDE
loop
        face_img = frame[y:y+h, x:x+w]
        face_resized = cv2.resize(face_img, (IMG_SIZE, IMG_SIZE))
        face_norm = face_resized / 255.0
        face_input = np.reshape(face_norm, (1, IMG_SIZE, IMG_SIZE, 3))

        pred = model.predict(face_input, verbose=0)[0]
        class_idx = np.argmax(pred)      # ← FIX: now INSIDE loop
        confidence = pred[class_idx] * 100
        label_text = f"{labels[class_idx]}: {confidence:.1f}%"

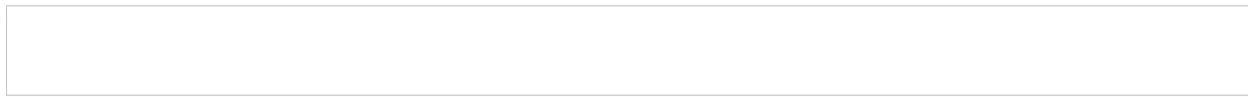
        # Green for Mask, Red for No Mask
        color = (0, 255, 0) if class_idx == 0 else (0, 0, 255)

        cv2.rectangle(frame, (x, y), (x+w, y+h), color, 2)
        cv2.putText(frame, label_text, (x, y-10),
                     cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

        cv2.imshow("Real-Time Mask Detection", frame)
        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

cap.release()
cv2.destroyAllWindows()

```



Task 2 Face Recognition Attendance System

Task Objective

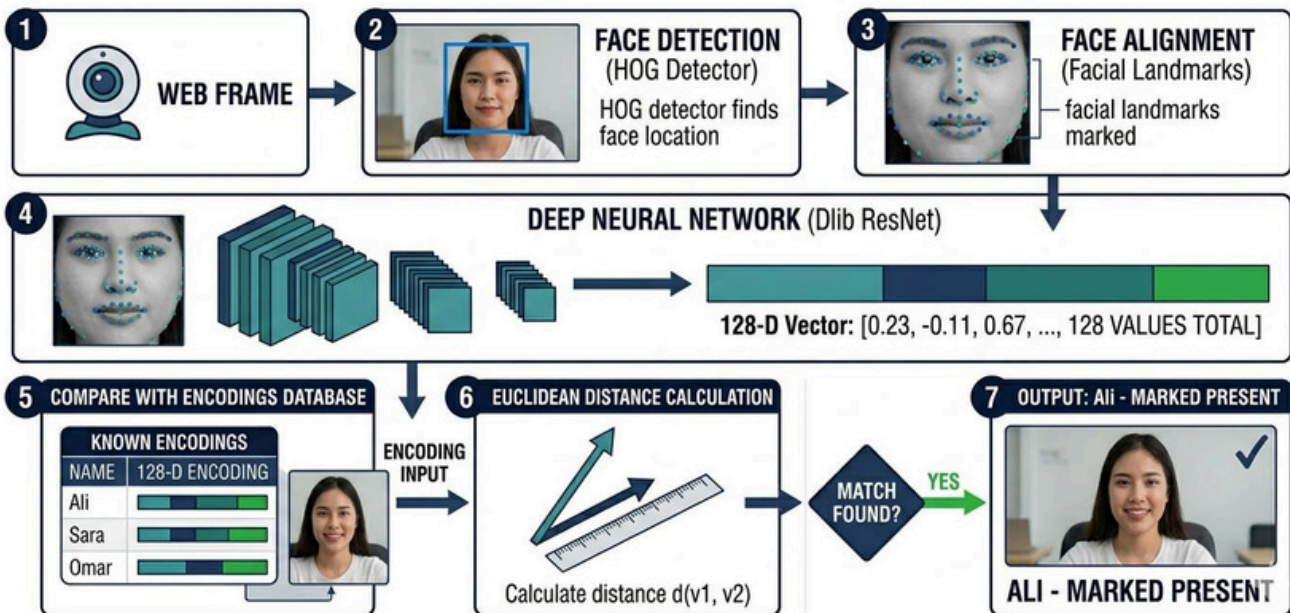
Build a face-recognition attendance system that uses a webcam to identify registered students and automatically logs their name, date, and time to a CSV file.

Learning Outcomes: Use pre-trained deep learning embeddings (dlib/face_recognition) · Implement real-world biometric identification · Automate record-keeping with Python.

How Face Recognition Works

Unlike Task 1 (training our own CNN), this task uses a pre-trained deep learning model (dlib's ResNet-based face encoder) that produces a 128-dimensional numerical vector called a face embedding for any face image. Faces of the same person produce very similar embeddings; different people produce distant embeddings. We compare embeddings using Euclidean distance.

COMPLETE FACE RECOGNITION FOR A STUDENT IN STUDENT



Installation CMake and dlib

Important: The face recognition library depends on ^{dlib}, which must be compiled from source. CMake is required for this compilation step on Windows.

Step 1 Install CMake

Download the Windows x64 Installer from <https://cmake.org/download/>. During installation, select 'Add CMake to the system PATH for all users'. After install, verify:

```
cmake --version
# Expected output: cmake version 3.29.x (or similar)
```

Step 2 Install dlib and face_recognition

In Anaconda Prompt with tfenv activated:

```
conda activate tfenv

# Install dlib (builds from source may take 5-10 minutes)
conda install -c conda-forge dlib

# Install face_recognition wrapper
```

```
conda install -c conda-forge face-recognition
```

```
#Verify both installed correctly  
python -c "import dlib; import face_recognition; \  
    print('dlib:', dlib.__version__); print('face_recognition: OK')"
```

Troubleshooting dlib Installation

- If conda install fails: try pip install dlib (may need Visual Studio Build Tools on Windows).
- Visual Studio Build Tools download: <https://visualstudio.microsoft.com/visual-cpp-build-tools/>
- Check that CMake version is 3.29+ before attempting dlib install.
- Confirm tfenv is active (you should see (tfenv) in your prompt) before installing.

Step 1 Prepare Known Faces Dataset

Create a folder called 'known_faces' and add photos of each student to be recognized:

```
known_faces/  
├── Ali.jpg          ← Single photo named exactly as the student's name  
├── Sara.jpg  
├── Ahmed_0.jpg      ← Multiple photos of same person increase accuracy  
├── Ahmed_1.jpg  
└── Fatima.jpg
```

Photo Guidelines for Best Accuracy

- Use clear, well-lit, front-facing photos similar to passport photos.
- One high-quality photo per student is sufficient; 2–3 photos improve robustness.
- File name = student name (no spaces; use underscore). E.g., 'Muhammad_Ali.jpg'
- Remove the extension when reading names: 'Muhammad_Ali.jpg' → 'Muhammad_Ali'.

Step 2 Full Attendance System Code

```
import cv2, face_recognition, os, numpy as np, csv  
from datetime import datetime
```



```

logged_today.add(name)
print(f" Attendance marked: {name} at {now.strftime('%H:%M:%S')}")

init_attendance_file()

# =====
# STEP 3: Real-time webcam recognition
# =====
cap = cv2.VideoCapture(0)
if not cap.isOpened():
    raise IOError("Cannot access webcam.")

print("Attendance System running. Press 'q' to quit.")

TOLERANCE = 0.5 # Lower = stricter matching (0.4-0.6 recommended)

while True:
    success, frame = cap.read()
    if not success:
        break

    # Downscale for faster processing (4x smaller = 16x faster detection)
    small = cv2.resize(frame, (0, 0), fx=0.25, fy=0.25)
    rgb_small = cv2.cvtColor(small, cv2.COLOR_BGR2RGB)

    # Detect all faces and compute their encodings
    face_locs = face_recognition.face_locations(rgb_small)
    face_encs = face_recognition.face_encodings(rgb_small, face_locs)

    for enc, loc in zip(face_encs, face_locs):
        # Compare current face against all known faces
        distances = face_recognition.face_distance(known_encodings, enc)
        best_idx = np.argmin(distances)

        if distances[best_idx] < TOLERANCE:
            name = known_names[best_idx]
            confidence = (1 - distances[best_idx]) * 100
            label = f"{name} ({confidence:.1f}%"
            color = (0, 255, 0) # Green for known person
            mark_attendance(name)
        else:
            label = "Unknown"
            color = (0, 0, 255) # Red for unknown

    # Scale coordinates back to full frame size
    top, right, bottom, left = [v * 4 for v in loc]
    cv2.rectangle(frame, (left, top), (right, bottom), color, 2)

```

```

cv2.putText(frame, label, (left, top - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)

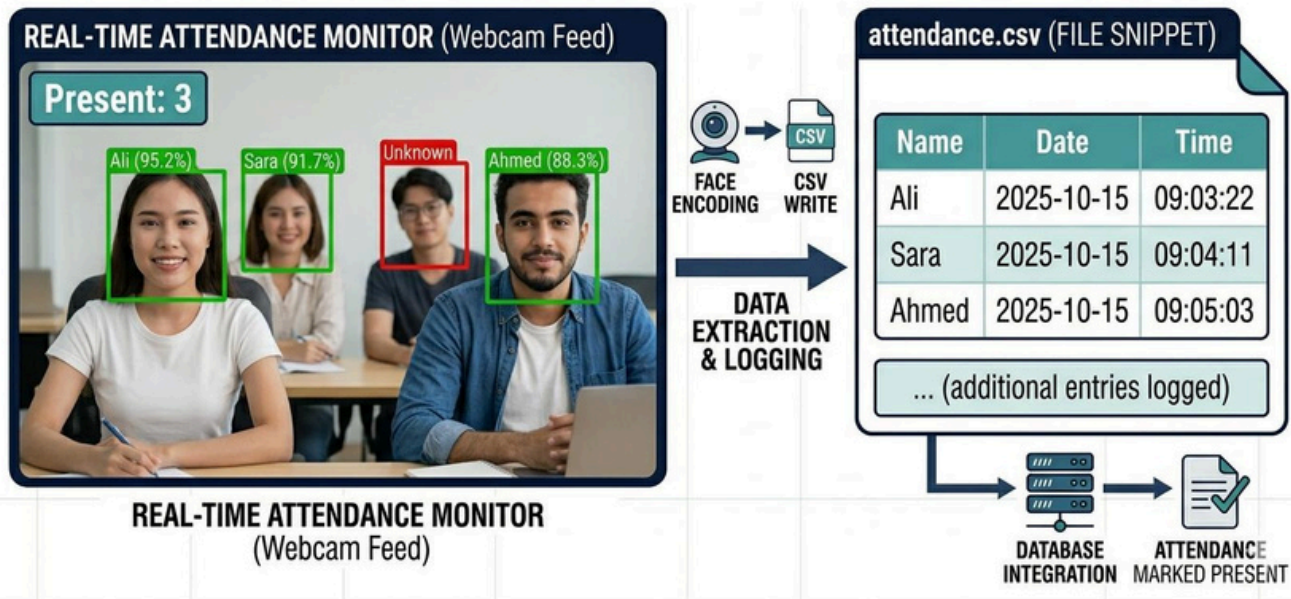
# Show count of logged students
cv2.putText(frame, f"Present: {len(logged_today)}", (10, 30),
            cv2.FONT_HERSHEY_SIMPLEX, 0.9, (255,255,0), 2)

cv2.imshow("Face Recognition Attendance System", frame)
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

cap.release()
cv2.destroyAllWindows()
print(f"Session ended. {len(logged_today)} student(s) marked present.")
print(f"Attendance saved to '{ATTENDANCE_FILE}'")

```

FACE-BASED ATTENDANCE AND CLASSIFICATION SYSTEM OUTPUT



Task #01

LAB TASKS

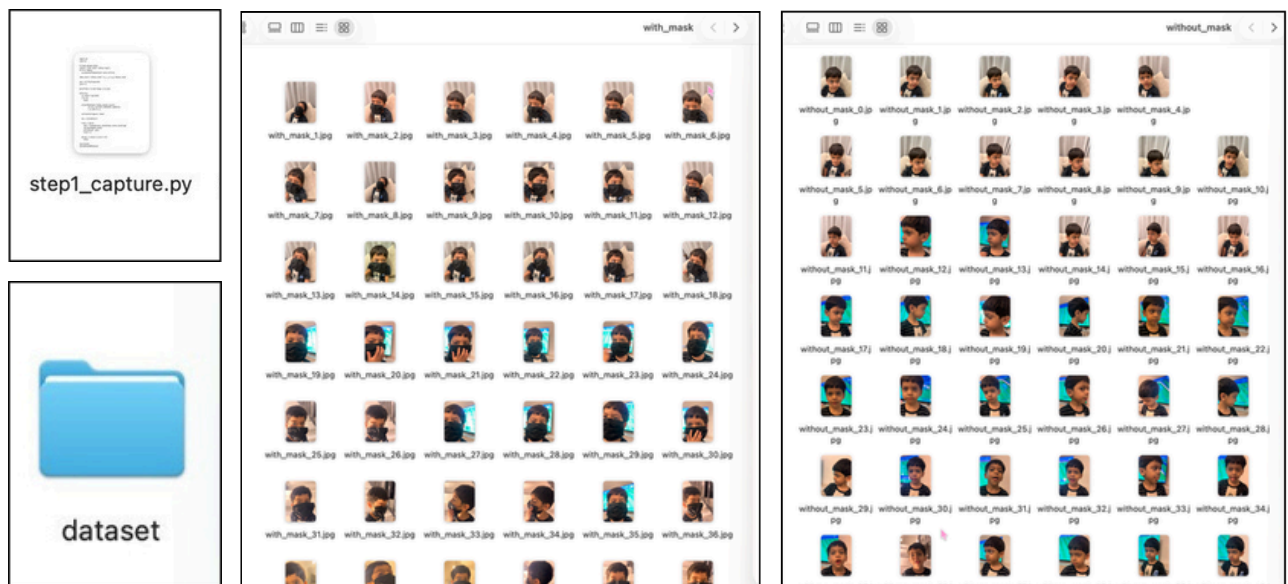
Task 1 Real-Time Mask Detection Using CNN

Task Objective

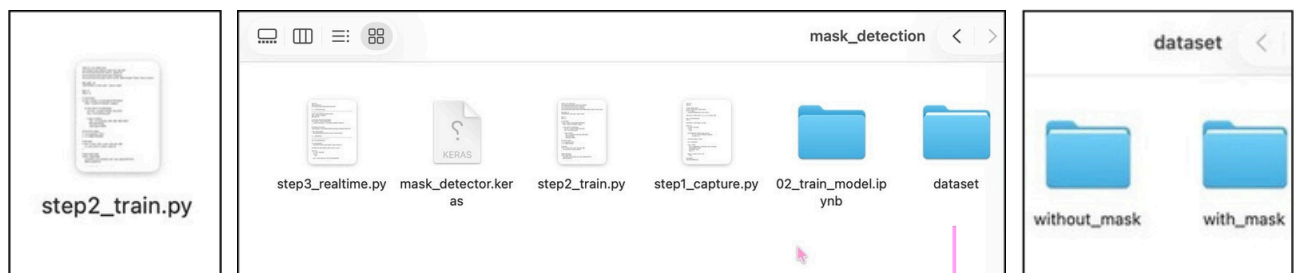
Train a CNN model to classify webcam frames as 'Mask' or 'No Mask' in real time. This simulates a practical computer vision application used in health safety monitoring systems.

Learning Outcomes: Build a CNN from scratch · Collect your own dataset · Train, save, and deploy a model · Apply real-time inference with OpenCV face detection.

Step 1 Dataset Collection



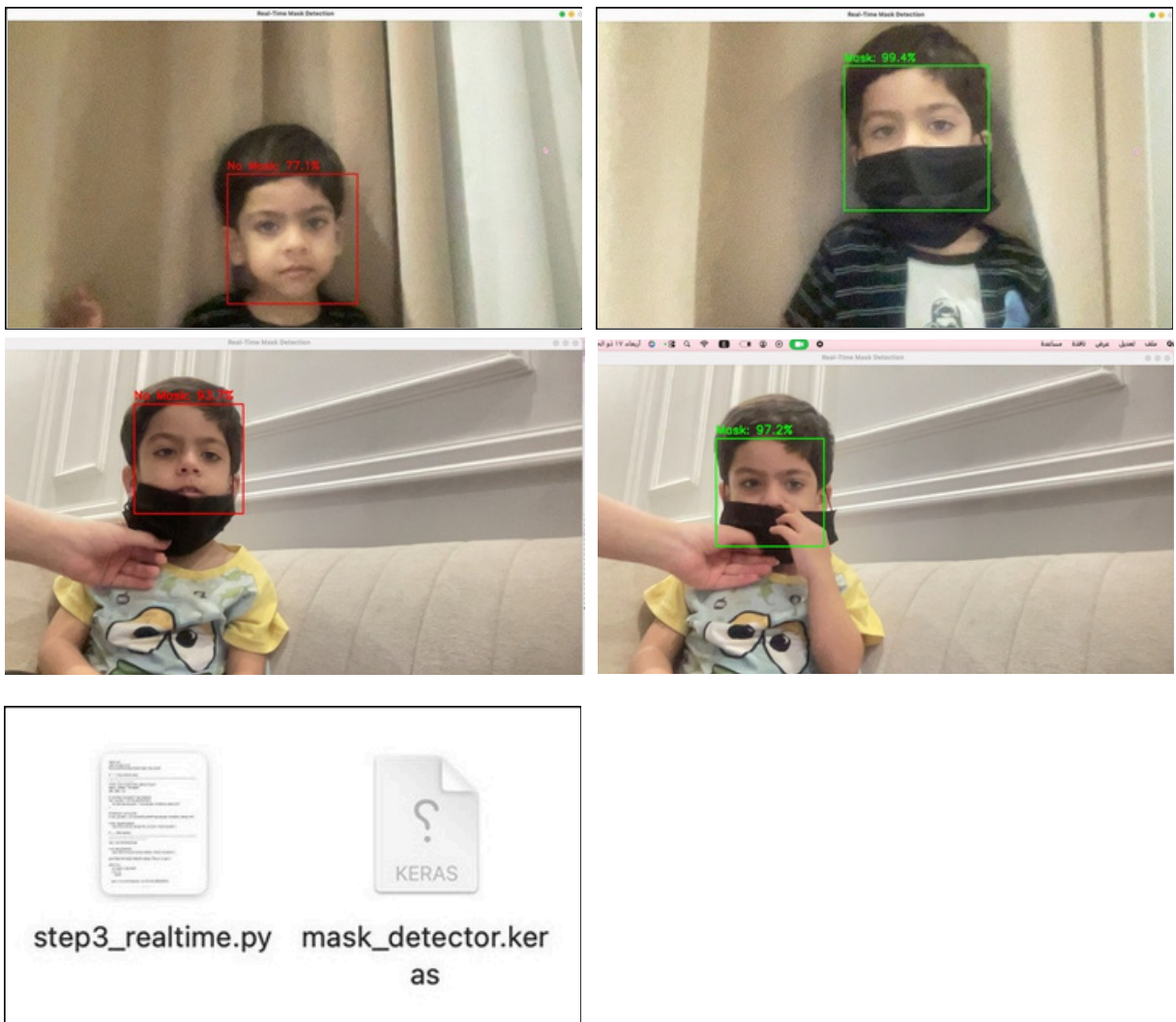
Step 2 Data Preprocessing & Model Training



Step 2 Data Preprocessing & Model Training

```
[maskenv] bushra@MacBook-Air-bshry mask_detection N python step2_train.py
/opt/anaconda3/envs/maskenv/lib/python3.10/site-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an 'input_shape'/'input_dim' argument to a layer. When using Sequential
models, prefer using an 'Input(shape)' object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
2026-06-02 01:58:48.165196: I metal_plugin/src/device/metal_device.cc:1154] Metal device set to: Apple M1
2026-06-02 01:58:48.165248: I metal_plugin/src/device/metal_device.cc:1296] systemMemory: 8.00 GB
2026-06-02 01:58:48.165248: I metal_plugin/src/device/metal_device.cc:313] maxCacheSize: 2.67 GB
2026-06-02 01:58:48.165458: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:385] Could not identify NUMA node of platform GPU ID 0, defaulting to 0. Your kernel may not have been built with NUMA support.
2026-06-02 01:58:48.165472: I tensorflow/core/common_runtime/pluggable_device/pluggable_device_factory.cc:271] Created TensorFlow device (/job:localhost/replica:0/task:0/device:GPU:0 with 0 MB memory) -> physical PluggableDevice (device: 0, name: METAL, pci bus id: <undefined>)
Epoch 1/10
2026-06-02 01:58:49.497827: I tensorflow/core/grappler/optimizers/custom_graph_optimizer_registry.cc:113] Plugin optimizer for device_type GPU is enabled.
5/5 ----- 5s 138ms/step - accuracy: 0.5375 - loss: 1.3623 - val_accuracy: 0.4000 - val_loss: 1.1152
Epoch 2/10
5/5 ----- 5s 31ms/step - accuracy: 0.5875 - loss: 0.9539 - val_accuracy: 0.4000 - val_loss: 0.6874
Epoch 3/10
5/5 ----- 5s 30ms/step - accuracy: 0.6875 - loss: 0.6397 - val_accuracy: 0.5500 - val_loss: 0.6420
Epoch 4/10
5/5 ----- 5s 31ms/step - accuracy: 0.6625 - loss: 0.5973 - val_accuracy: 0.7000 - val_loss: 0.5787
Epoch 5/10
5/5 ----- 5s 30ms/step - accuracy: 0.8875 - loss: 0.4792 - val_accuracy: 0.9000 - val_loss: 0.4279
Epoch 6/10
5/5 ----- 5s 35ms/step - accuracy: 0.8750 - loss: 0.3784 - val_accuracy: 0.9500 - val_loss: 0.3099
Epoch 7/10
5/5 ----- 5s 32ms/step - accuracy: 0.9375 - loss: 0.2432 - val_accuracy: 1.0000 - val_loss: 0.1534
Epoch 8/10
5/5 ----- 5s 29ms/step - accuracy: 0.9875 - loss: 0.1144 - val_accuracy: 1.0000 - val_loss: 0.0820
Epoch 9/10
5/5 ----- 5s 30ms/step - accuracy: 0.9875 - loss: 0.0763 - val_accuracy: 1.0000 - val_loss: 0.0689
Epoch 10/10
5/5 ----- 5s 30ms/step - accuracy: 0.9750 - loss: 0.0482 - val_accuracy: 1.0000 - val_loss: 0.0128
✓ Model trained and saved successfully!
[maskenv] bushra@MacBook-Air-bshry mask_detection N
```

Step 3 Real-Time Mask Detection



Task #02

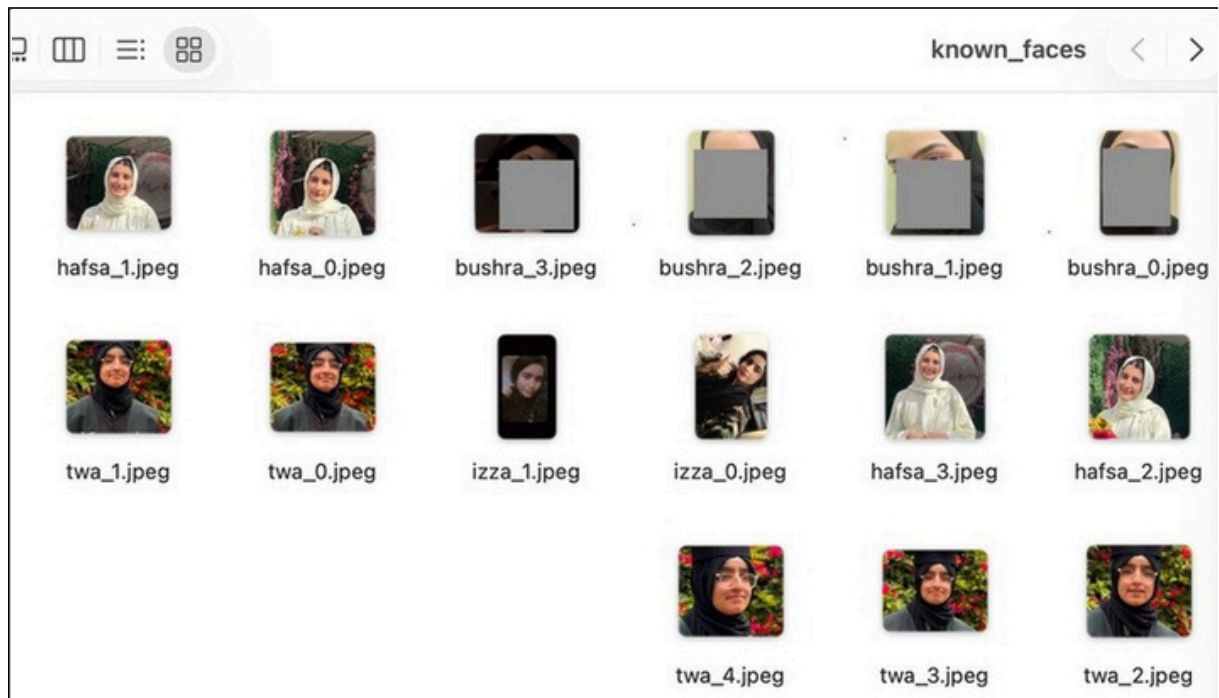
Task 2 Face Recognition Attendance System

Task Objective

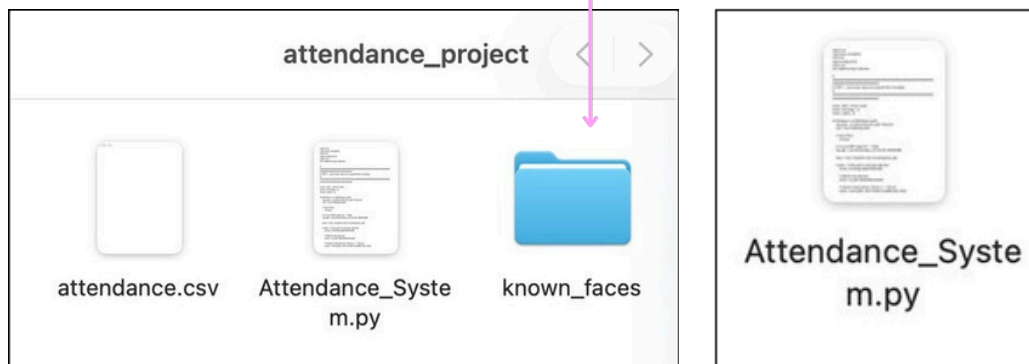
Build a face-recognition attendance system that uses a webcam to identify registered students and automatically logs their name, date, and time to a CSV file.

Learning Outcomes: Use pre-trained deep learning embeddings (dlib/face_recognition) · Implement real-world biometric identification · Automate record-keeping with Python.

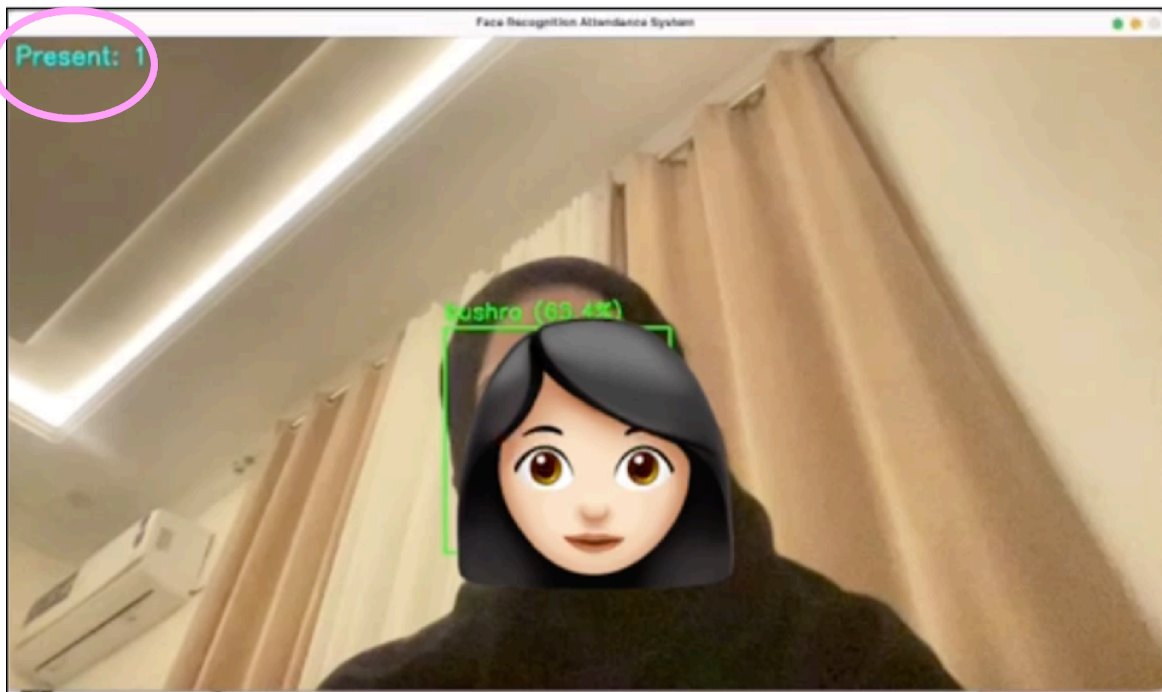
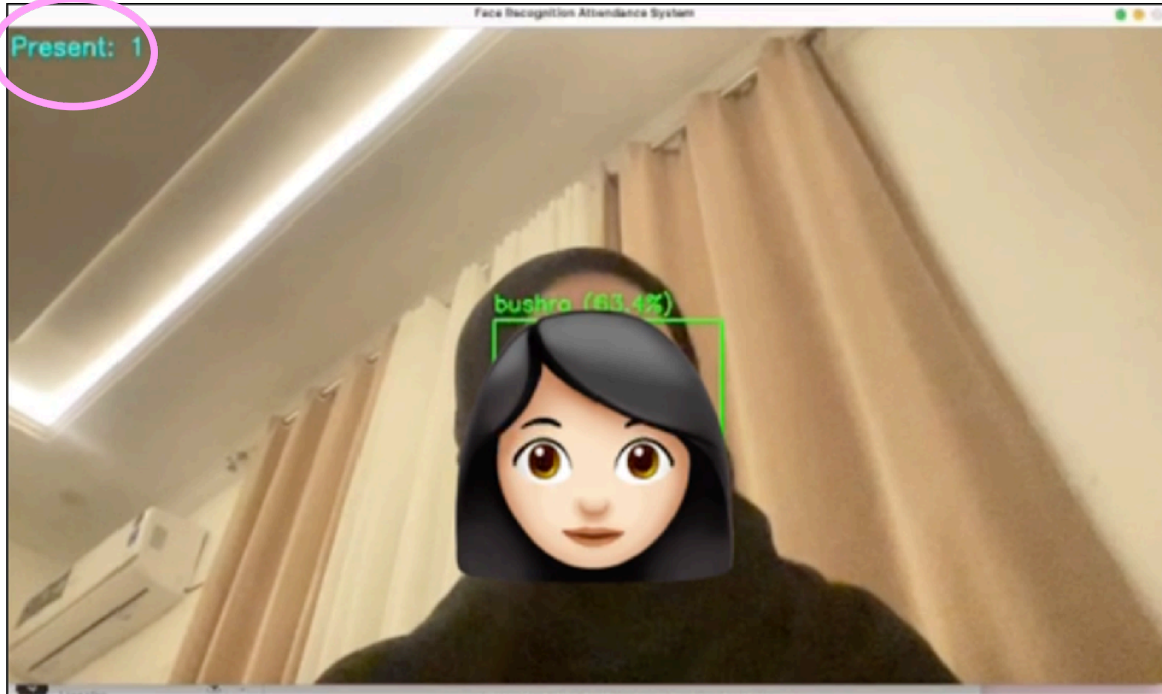
Step 1 Prepare Known Faces Dataset



Step 2 Full Attendance System Code



Step 2 Full Attendance System Code



```
(attendance_env) bushra@MacBook-Air-bshry attendance_project % conda activate attendance_env
python Attendance_System.py
Loaded 15 known face(s): ['izza', 'bushra', 'twa', 'twa', 'bushra', 'izza', 'hafsa', 'hafsa', 'twa', 'hafsa', 'hafsa', 'bushra', 'twa', 'twa', 'bushra']
Attendance System running. Press 'q' to quit.
Attendance marked: bushra at 01:01:32
Session ended. 1 student(s) marked present.
Attendance saved to 'attendance.csv'
```

attendance.csv X

Users > bushra > Desktop > attendance_project > attendance.csv

```
1 Name,Date,Time
2 bushra,2026-06-03,01:01:32
```

International Islamic University Islamabad
Faculty of Engineering and Technology
Department of Electrical Engineering

Artificial Intelligence & Machine Learning LAB

Lab No. 14

Implementing Convolutional Neural Network (CNN) for Edge-Based Mask Detection and Face Recognition

Name: Bushra Nazir

Reg. No.: 1071-F22

S. No.	Criteria	Allocated Percentage	Level 1 (0%)	Level 2 (25%)	Level 3 (50%)	Level 4 (75%)	Level 5 (100%)	Marks Obtained
(A) PSYCHOMOTOR (To be judged in the lab during experiment)								
1	Practical Implementation OR Connectivity/Arrangement of Equipment	5	Absent	Degree of errors in applying procedural knowledge or Degree of errors in arrangement of equipment:				
			0	With several critical errors, Imcomplete and Not Neat	With few errors, Incomplete and not Neat	With some errors, Complete but not Neat	Without errors, Complete and Neat	
				1.25	2.5	3.75	5	
2	Use of Equipment OR Use of Simulation/Programming Tool	2	Absent	Use of tools, equipment and materials with:				
			0	Limited Competence	Some Competence	Considerable Competence	Good Competence	
				0.5	1	1.5	2	
3	Safety (Optional)	0	Absent	Degree of reminders to follow safety procedure with:				
			0	Rarely Follow Safety Rules	Needs Reminders	Needs minor Reminders	Follows Safety Rules	
				0	0	0	0	
	SUB-TOTAL (PT)	7	SUB-TOTAL MARKS OBTAINED (PO)					
(B) COGNITIVE (To be judged on the copy of experiment submitted)								
4	Data Record, Analysis and Evaluation OR Algorithm Design	1	Absent	Data record, Analysis and Evaluation or Designed Algorithm is:				
			0	Incorrect /Incomplete	Complete with some errors	Complete with few errors	Complete and Accurate	
				0.25	0.5	0.75	1	
5	Procedural Awareness (Optional)	0	Absent	Procedural awareness of the completed task is:				
			0	Inappropriate	Appropriate with some errors	Appropriate with few errors	Appropriate	
				0	0	0	0	
	SUB-TOTAL (CT)	1	SUB-TOTAL MARKS OBTAINED (CO)					
(C) AFFECTIVE (To be judged in the lab during experiment)								
6	Level of Participation & Attitude to Achieve Individual/Group Goals	2	Absent	Degree of positive interaction as an individual or within a group:				
			0	Rare interaction	Some interaction	Acceptable interaction	Good interaction	
				0.5	1	1.5	2	
	SUB-TOTAL (AT)	2	SUB-TOTAL MARKS OBTAINED (AO)					

Total Marks (PT + CT + AT) :

10

Marks Obtained (PO + CO + AO) :

Instructor